

برنامه‌سازی پیشرفته

تمرین کامپیوتری شماره ۲

مدرس: رامتین خسروی



طراحان: صادق صمدی، سپنتا غنودی، مهدیس
میرزایی، رضا چهرقانی، پارسا سعیدنیا، نیما ادیب،
فرشته باقری، سامان رکرك

مهلت تحویل: جمعه ۴ اردیبهشت ۱۴۰۵، ساعت ۲۳:۵۹

مقدمه

این تمرین برای آشنایی با برنامه‌نویسی بازگشتی طراحی و در قالب سه سوال مجزا تهیه شده است که پیشنهاد می‌شود برای درک بهتر مفاهیم برنامه‌نویسی بازگشتی، زمان کافی را برای پاسخ دادن به آن‌ها اختصاص دهید. توجه کنید که پرسش‌ها حتماً باید به روش بازگشتی حل شوند، هر چند ممکن است روش‌های دیگری نیز برای حل آن‌ها وجود داشته باشد.

تمرین اول: اسکندر باهوش



روزی روزگاری در سرزمین شکرستان در یکی از روزهای زیبای بهار، ننه قمر به همراه اسکندر در حال رفتن به خانه بهلول بودند. در طی مسیر اسکندر به ننه قمر از کشف جدیدش خبر داد. اسکندر متوجه شده بود بعضی از جملات را می‌توان از دو طرف به یک صورت خواند. به عبارتی از چپ به راست یا راست به چپ یک جمله است و قرینه هست. ننه قمر در حالی که به نوه باهوش خود افتخار می‌کرد به او گفت: اسکندر ننه بیا بهلول رو امتحان کنیم. به بهلول جملات انگلیسی بدیم و بهش بگیریم قرینه‌ای هست یا نه؟ اسکندر خوشحال شد و خندید. حال شما باید به بهلول کمک کنید و بگویید آیا جملاتی که ننه قمر و اسکندر به او می‌گویند قرینه‌ای هست یا خیر؟

ورودی

در خط اول ورودی تعداد جملات به شما داده می‌شود. سپس در n خط بعدی، در هر خط یک جمله به عنوان ورودی داده می‌شود که جملات اسکندر است. دقت کنید این عبارت می‌تواند شامل فاصله یا هر کاراکتر دیگری باشد اما مبنای کار اسکندر فقط حروف انگلیسی است. به عبارتی دیگر این ورودی می‌تواند شامل هر چیزی باشد اما شما فقط حروف انگلیسی را باید در نظر بگیرید. دقت کنید شما در روش حل خود باید سایر کاراکترها را نیز به صورت بازگشتی هندل کنید. به عبارتی اگر رشته را تغییر دهید حتی اگر ادامه حل شما بازگشتی باشد هیچ نمره‌ای به شما تعلق نمی‌گیرد. همچنین توجه کنید بین حرف کوچک و بزرگ تفاوتی وجود ندارد. برای درک بهتر به نمونه‌ها توجه کنید.

خروجی

خروجی شما فقط کلمه yes یا no است که نشان می‌دهد جمله اسکندر قرینه‌ای هست یا خیر.

نمونه ورودی	نمونه خروجی
never odd or even	yes
A man, a plan, a canal: Panama!	yes
salma	no
Hello, World!	no
123??@#abcCBA	yes

تمرین دوم: بازگشایی!

دزدان شکرستان که به دنبالی راهی امن برای پایه ریزی نقشه های شوم خود بودند، تصمیم گرفتند راهی امن برای ارتباط بین خود، مانند ماشین انیگما اختراع کنند که متون را به صورت رمزنگاری شده منتقل کنند. حال پیامی به دست آنها رسیده که حاوی نقشه برای حمله امشب است. اما دستگاه او خراب شده و قادر به رمزگشایی پیام نیست. از آنجایی که او مطمئن است الگوریتم دستگاه اصلی با روابط بازگشتی نوشته شده بود، از شما می‌خواهد تا با نوشتن الگوریتمی بازگشتی، رمز را برای او بر ملا کنید تا بتواند به نقشه حمله امشب برسد! (از آنجایی که او به یاد دارد الگوریتم اصلی بازگشتی است، روش دیگری را از شما قبول نمی‌کند!). هر رمز شامل یک رشته S به صورت رمزنگاری شده است. این رشته شامل حروف الفبای انگلیسی، کروشه¹ و اعدادی مثبت است. فرمت قرارگیری کروشه و عددی مانند k به صورت $k[S]$ می‌باشد. پیام رمز به این صورت محاسبه می‌شود که کاراکترهای خارج $[]$ به صورت مستقیم در خروجی نمایش داده می‌شوند و زیر رشته داخل $[]$ به تعداد k بار (عددی که پشت کروشه قرار می‌گیرد) پشت هم تکرار می‌شوند. حال به این شخص کمک کنید تا بتواند با نقشه دزدی امشب هماهنگ شود!

محدودیت

$$1 \leq S.length \leq 30$$

ورودی

در یک خط یک رشته S برای رمزگشایی (بازگشایی!) داده می‌شود.

¹ Square brackets

خروجی

خروجی شامل یک رشته شامل پیام به دست آمده از رمزگشایی S باشد.

نمونه ورودی ۱
$3[a]2[bc]$
نمونه خروجی ۱
aaabcbc

نمونه ورودی ۲
$b3[a2[c]]$
نمونه خروجی ۲
baccaccacc

تمرین سوم: اسکندر در کوچه و بازار

اسکندر ۲۱ ساله که در قطعی اینترنت حوصله‌اش سر رفته بود، تصمیم گرفت برای گذران وقت به دوستان ۱۰ ساله‌اش که در کوچه بازی می‌کردند، بپیوندد. از آن‌جایی که اسکندر به توانایی‌های خود برای برد در بازی اطمینان نداشت، تصمیم گرفت ابتدا برای مدتی از پنجره، بازی آن‌ها را مورد مطالعه قرار دهد و پس از یافتن استراتژی برد، به آن‌ها بپیوندد. او پس از مدتی متوجه شد که بازی به این صورت است: بازیکنان در یک جدول مربعی 7×7 قرار گرفته‌اند. از آن‌جایی که زمین این بازی خاکی است، در زمین ۱۶ چاله وجود دارد که بازیکنان نمی‌توانند روی آن قرار بگیرند. در ابتدای بازی، باقی خانه‌های زمین یا خالی است یا دقیقاً یک بازیکن در آن قرار دارد. چاله‌های موجود در زمین همواره در خانه‌های مشخصی قرار دارند و مکان آن‌ها تغییر نمی‌کند و به صورت زیر است:

نمونه نقشه چاله‌های زمین
XX000XX XX000XX 0000000 0000000 0000000 XX000XX XX000XX

که خانه‌های X چاله‌ها هستند و خانه‌های O خالی هستند که بازیکنان می‌توانند در آن‌جا قرار بگیرند. برای افزایش هیجان در این بازی، در هر مرحله بازیکنان می‌توانند از روی دقیقاً یک بازیکن که در یکی از خانه‌های مجاور آن قرار دارد بپرند و روی خانه‌ی خالی بعد از آن قرار بگیرند. این خانه‌ی خالی در فاصله‌ی ۲ خانه از بازیکن قرار دارد. بازیکن نمی‌تواند از روی خانه‌ی خالی یا چاله‌ها بپرد و حتماً باید بازیکنی در خانه‌ی مجاور او قرار داشته باشد تا بازیکن بتواند عمل پرش را انجام دهد. همچنین، خانه‌ای که بازیکن پس از پرش روی آن قرار می‌گیرد، باید خالی باشد. هر بار بازیکنی که از روی آن پریده می‌شود، حذف و از بازی خارج شده و خانه‌ای که در آن قرار داشت خالی می‌شود. در نهایت اگر فقط یک بازیکن در جدول بماند و آن بازیکن در خانه‌ی وسط جدول باشد، همه‌ی بازیکنان برنده‌ی بازی خواهند بود و اگر بیشتر از یک بازیکن باقی بماند اما دیگر نتوانیم پرشی انجام دهیم، یا این‌که فقط یک مهره در جدول بماند اما در خانه‌ی وسط جدول نباشد، همه‌ی بازیکنان بازنده خواهند بود. اسکندر می‌خواهد روشی پیدا کند که به ازای هر چینش جدولی، روشی برای برد بیابد یا اعلام کند که روشی برای برد وجود ندارد، تا به عنوان یک قهرمان وارد بازی‌های کوچه شود. لازم به ذکر است که راه‌حل شما باید حتماً با استفاده از روش backtracking مسئله را حل کند.

ورودی

ورودی برنامه شامل ۷ خط خواهد بود که هر خط شامل ۷ حرف است:

- حرف X به این معناست که خانه‌ی متناظر یک چاله است.
- حرف N به معنی وجود یک بازیکن در آن خانه است.
- حرف O به این معناست که خانه خالی است.

دقت کنید که در ورودی هیچ فاصله‌ای بین حروف یک خط وجود ندارد.

نمونه ورودی ۱
XX000XX
XX000XX
0000000
00N00N0
0000NNO
XXON0XX
XX000XX

خروجی

برای چاپ مراحل، ابتدا به تصویر زیر که نام هر خانه‌ی جدول را مشخص می‌کند، توجه کنید:

نام خانه‌های جدول
A1 A2 A3 A4 A5 A6 A7
B1 B2 B3 B4 B5 B6 B7
C1 C2 C3 C4 C5 C6 C7
D1 D2 D3 D4 D5 D6 D7
E1 E2 E3 E4 E5 E6 E7
F1 F2 F3 F4 F5 F6 F7
G1 G2 G3 G4 G5 G6 G7

توجه شود که منظور از خانه وسط جدول، D4 است. هر مرحله را به صورت ترکیبی از خانه‌ی مبدأ و جهت حرکت (UP, DOWN, LEFT, RIGHT) بازیکن نمایش می‌دهیم. برای مثال، فرض کنید یک بازیکن در خانه‌ی سوم سمت چپ و در ردیف اول قرار دارد (خانه‌ی A3) و خانه‌ی C3 هم خالی است. در این صورت اگر بخواهیم مهره‌ای

که در خانه‌ی A3 قرار دارد از روی مهره‌ای که در خانه‌ی B3 قرار دارد بپرد و وارد خانه‌ی C3 شود، مرحله را به این صورت در خروجی نمایش می‌دهیم: A3 DOWN

نمونه خروجی ۱
E6 LEFT F4 UP D3 RIGHT D6 LEFT

نمونه ورودی ۲
XX000XX XX000XX 0000000 00N0000 0000000 XX0N0XX XX000XX
نمونه خروجی ۲
Loser

نکات و نحوه تحویل

- برنامه شما باید در سیستم عامل لینوکس و با مترجم g++ با استاندارد c++20 ترجمه و در زمان معقول برای ورودی‌های آزمون اجرا شود.
- تحویل این تمرین در سامانه ای‌لرن انجام می‌شود. کدهای خود را در فایلی به نام A2-<SID>.zip قرار دهید که در نام آن SID شماره دانشجویی شماست. سپس این فایل را در محل مشخص شده در ای‌لرن بارگذاری کنید. برای مثال اگر شماره دانشجویی شما 810100000 باشد، نام فایل بارگذاری شده توسط شما باید A2-810100000.zip باشد. دقت داشته باشید که از کاراکتر خط فاصله² (-) استفاده کنید و از خط زیرین³ (_) استفاده نکنید.
- کد هر سوال را در یک فایل مجزا با فرمت Q#.cpp قرار دهید. برای مثال نام فایل حاوی کد پاسخ سوال ۱ می‌شود Q1.cpp. سپس فایل‌های کدهای خود را در صفحه اول فایل فشرده قرار دهید به گونه‌ای پس از خارج کردن از حالت فشرده، کدهای شما در پوشه‌ای قرار نگیرند.
- دقت کنید که عدم رعایت ساختار گفته شده در آپلود باعث کسر ۱۰ درصد از نمره شما خواهد شد.
- بخش مهمی از ارزیابی برنامه شما درستی عملکرد آن است. بنابراین به اندازه کافی ورودی‌های آزمایشی طراحی کنید تا درستی خروجی در حالت‌های مختلف آزموده شود و دقت کنید که اگر برنامه شما با مثال‌های داده شده درست کار کند لزوماً به معنای درستی در تمام سناریوها نیست.
- درستی برنامه شما از طریق آزمون‌های خودکار سنجیده می‌شود. به این ترتیب، لازم است خروجی تولید شده از نظر بزرگی و کوچکی حروف، رعایت فاصله‌ها، عدم وجود خروجی‌های اضافه، ... دقیقاً مانند نمونه‌های داده شده باشد. بنابراین پیشنهاد می‌شود که با استفاده از ابزارهایی مانند diff فرمت خروجی برنامه خود را با خروجی‌هایی که در اختیارتان قرار داده شده است مطابقت دهید.
- هدف این تمرین یادگیری شماست. لطفاً تمرین را خودتان انجام دهید. در صورت کشف تقلب مطابق سیاست درس با آن برخورد خواهد شد.

² Hyphen

³ Underscore

نمرات

- تمیزی کد

- رعایت کردن نام‌گذاری صحیح و انسجام⁴
- عدم وجود کد تکراری
- رعایت دندانه‌گذاری⁵
- استفاده صحیح از متغیرهای ثابت به جای Magic Value-ها
- ساختاردهی کد در قالب توابع کوتاه که فقط یک کار را انجام می‌دهند

- درستی کد

- آزمون‌های خودکار

- طراحی بازگشتی

- پیاده‌سازی الگوریتم بازگشتی و عدم استفاده از حلقه (به جز موارد مجاز)

دقت کنید که موارد ذکر شده لزوماً کل نمره شما را تشکیل نمی‌دهند و ممکن است با تغییراتی همراه باشند.

⁴ Consistency

⁵ Indentation